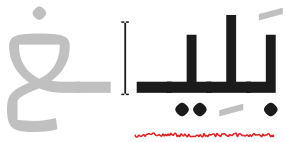
Cairo University
Faculty of Engineering
Department of Computer Engineering



Baligh

Arabic Writing Assistant

A Graduation Project Report Submitted

to

Faculty of Engineering, Cairo University

in Partial Fulfillment of the requirements of the degree

of

Bachelor of Science in Computer Engineering.

Presented by

Akram Hany Karam Sallam

Supervised by

Dr. Ayman AboElhassan

July, 2026

Abstract

Arabic digital writing still suffers from a shortage of integrated tools that can correct Modern Standard Arabic text accurately, respond in real time, and explain their feedback in a way that helps users learn. Existing tools often focus on surface spelling fixes or black-box suggestions without exposing the grammatical reasoning behind the correction. Baligh addresses this problem by providing an Arabic writing assistant that combines grammatical error detection, grammatical error correction, spelling-aware editing, automatic vowelization (Tashkeel), next-word suggestion, and explanatory feedback in a single user-facing system. The project aims to support students, educators, and Arabic content writers by improving writing quality while also making the correction process more understandable and educational.

The implemented system follows a modular pipeline. A preprocessing layer performs normalization, word-boundary handling, segmentation, and morphological analysis and disambiguation. On top of that, Baligh integrates a grammatical error detection engine that fuses rule-based, lexicon-based, and machine-learning detectors, then passes its findings to correction components built around ontology-guided reasoning, dictionary lookup, text-editing models, and automatic vowelization support. In parallel, a next-word suggestion module supports both next-word prediction and word auto-completion. The final system highlights issues, proposes corrections, returns ranked suggestions, generates Tashkeel when needed, and presents grammar-oriented explanations to the user in an interactive workflow.

The project outputs include the preprocessing pipeline, grammatical error detection and correction services, the next-word suggestion subsystem, and a working user application. Testing and validation relied on automated unit and integration tests together with evaluation on established Arabic linguistic resources, including QALB14, QALB15, and ZAEBUC. Overall, Baligh demonstrates that a modular and explainable Arabic writing assistant can combine linguistic processing and machine learning into a practical platform for improving Arabic writing quality.

Contents

List of Figures

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
GEC	Grammatical Error Correction
GED	Grammatical Error Detection
JSON	JavaScript Object Notation
KSR	Keystroke Saving Rate
LSTM	Long Short-Term Memory
MRR	Mean Reciprocal Rank
MSA	Modern Standard Arabic
NLP	Natural Language Processing
NWP	Next-Word Prediction
NWS	Next-Word Suggestion
OOV	Out-Of-Vocabulary
WAC	Word Auto-Completion

Chapter 1: Introduction

This chapter introduces the project, justifies the need for it, states the project outcomes, and explains the organization of the report.

1.1. Motivation and Justification

Modern Standard Arabic (MSA) digital writing still lacks broadly available tools that combine grammatical support, writing speed, and educational value in a single user experience. While writing assistants for other languages provide immediate correction and prediction support, Arabic users often rely on tools that address only isolated spelling mistakes, provide limited grammatical help, or return suggestions without clarifying the linguistic reasons behind them. This gap is especially important in MSA, where accurate writing is influenced by rich morphology, orthographic ambiguity, and context-sensitive grammatical structures.

These characteristics make Arabic Natural Language Processing (NLP) demanding to model and serve in real-time. A useful writing assistant must process text carefully, understand partially typed input, and return feedback quickly. It must also present corrections in an explainable way rather than functioning as a black-box system. Baligh is motivated by this need for an integrated Arabic writing assistant that combines preprocessing, grammatical error detection (GED), grammatical error correction (GEC), and next-word prediction (NWP) / word auto-completion (WAC) in one coherent workflow.

1.2. Project Overview

Baligh is designed to assist individuals and organizations that produce Arabic text and require high grammatical accuracy. The system targets students, academic writers, and content creators. It features a modular pipeline: a preprocessing layer handles normalization and morphological analysis; a GED engine detects errors using rule-based, pattern-based, and machine learning subsystems; a GEC module proposes ontology-guided corrections; and a Next-Word Suggestion (NWS) module provides real-time predictions. Figure 1.1 illustrates this high-level modular architecture.

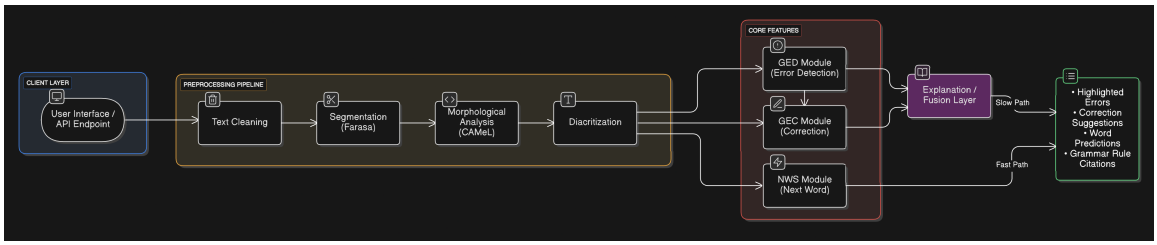


Figure 1.1: *High-level modular architecture of the Baligh system.*

1.3. Individual Role Overview

My role on the Baligh project was the design and implementation of two core layers:

1. **The Preprocessing Pipeline:** The first layer in the pipeline, responsible for text normalization, word boundary detection (mode switching), tokenization/segmentation using Farasa, and morphological analysis and disambiguation using CAMeL Tools.
2. **The Next-Word Suggestion (NWS) Subsystem:** A high-frequency, low-latency concurrent path that provides word suggestions during typing. It includes a three-tier cache (static idioms, famous phrases, and user LRU pattern caches), an N-gram Kneser-Ney statistical model, a 2-layer LSTM PyTorch neural network with SentencePiece tokenization, a character-level N-gram model, and a hybrid blending orchestrator.

1.4. Document Organization

The remainder of this report is organized as follows: Chapter 2 details my personal role, responsibilities, assigned tasks, and project timeline. Chapter 3 summarizes my knowledge acquisition phase, covering paper reviews and self-learning. Chapter 4 presents my core individual contributions in preprocessing and next-word suggestion, alongside evaluation and validation. Chapter 5 discusses the challenges encountered, lessons learned, and proposed future enhancements.

Chapter 2: Personal Role and Responsibilities

This chapter describes my specific responsibilities, individual tasks, and the project timeline.

2.1. Team Responsibilities

As a core member of the Baligh development team, my responsibilities included designing and documenting the input/output boundaries between pipeline layers. I worked closely with my teammates to establish robust, structured Pydantic schemas and MD data contracts, specifically authoring `preprocessing-contract.md` and `nws-contract.md`. I was also responsible for setting up unit and integration testing frameworks for the preprocessing pipeline and the next-word suggestion service, ensuring their reliability and correct feature integration.

2.2. Assigned Tasks

My specific technical tasks on the project were:

- **Conducting a market survey and product study:** Researching existing writing assistants and competitors to analyze their features, document their flaws, and define the target requirements and feature scope for Baligh.
- **Conducting literature review and architectural design for NWS:** Performing an extensive review of research papers on Arabic next-word prediction and autocompletion architectures to draft the initial system design.
- **Designing and implementing the text preprocessing pipeline:** Responsible for Unicode normalization, character offset mapping (mapping normalized indexes back to original text), word boundary splitting, and integrating the Farasa segmenter and CAMeL Tools morphological analyzer into a unified orchestrator.
- **Developing the Next-Word Suggestion (NWS) caching layer:** Authoring the three-tier cache manager to support static idioms, famous collocations, and user-specific LRU pattern caches.

- **Building the statistical language models:** Implementing the Modified Kneser-Ney (MKN) word N-gram model for next-word prediction (NWP) and a character N-gram backoff model for prefix completion (WAC).
- **Developing the neural next-word prediction model:** Training and integrating a PyTorch 2-layer LSTM language model using SentencePiece subword tokenization and implementing autoregressive beam search decoding with length normalization.
- **Implementing the hybrid orchestrator and scoring blend:** Creating the confidence-weighted blending module to dynamically mix and normalize predictions from statistical and neural sources.

2.3. Timeline and Deliverables

The project was executed in five major phases. Table 2.1 outlines the timeline, tasks, and deliverables.

Table 2.1: *Individual Project Timeline and Deliverables*

Phase	Tasks	Deliverable
Phase 1 (Research)	Literature survey of Arabic NLP tools, LMs, and segmenters.	Literature review draft.
Phase 2 (Design)	Designing API contracts and schemas for preprocessing and NWS.	`preprocessing-contract.md`, `nws-contract.md`.
Phase 3 (Preprocessing)	Implementing Unicode normalizer, offset maps, Farasa, and CAMeL Tools wrappers.	Preprocessing service module and integration tests.
Phase 4 (NWS Modeling)	Training LSTM, building Kneser-Ney word/character models, and caches.	Model checkpoints, cache loaders, and NWS engine.
Phase 5 (Verification)	Evaluating model accuracies, response latencies, and pipeline testing.	Final evaluation reports and passing test suite.

Chapter 3: Knowledge Acquisition and Technical Preparation

This chapter discusses the scientific literature and self-learning materials investigated to prepare for the implementation.

3.1. Investigated Papers

To design the preprocessing and next-word suggestion modules, I surveyed key papers in Arabic NLP and language modeling, focusing on the research papers in my local repository:

- **Next-Word Prediction and Neural Language Models:**
  - **Revealing the Next Word and Character in Arabic [6]:** Investigated its hybrid blend of Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNNs) for predicting characters and next words.
  - **Character-Aware Neural Language Models [8]:** Analyzed how character-level convolutional neural networks can be used as inputs to recurrent neural networks (LSTMs), allowing the system to handle out-of-vocabulary (OOV) words by exploiting character patterns.
  - **Telemedicine Predictive Text System [13]:** Studied its deep learning architecture for predictive text suggestions in Arabic, emphasizing model size vs. latency.
- **Statistical Language Models and Smoothing:**
  - **KenLM Language Model Querying [5]:** Studied optimized data structures (Probing and Trie) for fast, low-memory N-gram language model queries.
  - **Kurdish Next-Word Prediction [14]:** Analyzed next-word prediction based on N-gram backoff models for morphologically rich Kurdish dialects (Sorani and Kurmanji), evaluating Kneser-Ney and Katz backoff.
  - **Stanford NLP Course Notes on N-gram Models:** Examined theoretical foundations of maximum likelihood estimation (MLE), perplexity evaluation, and Modified Kneser-Ney smoothing algorithms.

3.2. Self-Learning Materials

In addition to research papers, I completed the following courses, documentation studies, and technical guides:

- **College Machine Learning course by Dr. Yahia Zakaria:** Strengthened the theoretical background behind statistical modeling, probability estimation, and evaluation metrics.
- **College Natural Language course by Dr. Sandra Wahid:** Reinforced language-processing concepts most relevant to Arabic NLP, including language modeling, smoothing techniques, and tokenization.
- **Farasa and CAMeL Tools Documentation:** Reviewed official software documentation, installation guides, and API references for the Farasa segmenter and CAMeL Tools suite to understand how they work internally and how to integrate them.

3.3. Relevance to the Project

This preparation directly shaped the architectural design of Baligh. Investigating Farasa and CAMeL Tools highlighted the mutual dependency between diacritization and morphological analysis, leading to the implementation of a joint morphological disambiguation approach. Reviewing character-aware language models and tries influenced the design of our character N-gram completion system to support out-of-vocabulary prefix matching in word auto-completion (WAC).

Chapter 4: Individual Contributions

This chapter presents my technical contributions to the preprocessing pipeline and the next-word suggestion subsystem.

4.1. Contribution 1: Text Preprocessing Pipeline and Morphological Analysis

The Preprocessing Pipeline is the entry point of the Baligh backend, responsible for parsing, cleaning, and extracting linguistic features from raw text before downstream GED, GEC, or NWS modules are invoked.

4.1.1. Unicode Normalization and Character Mapping

To ensure consistent processing, raw text typed by users goes through a normalization and cleaning stage. I designed and implemented this pipeline using the following steps:

- **Unicode NFKC Normalization:** Normalizes the input text using the Unicode NFKC standard to canonicalize varying character codepoints and ligature representations into standard forms.
- **Text Cleaning and Tatweel Stripping:** Strips redundant tatweel character segments (ـ) and consolidates consecutive whitespaces into single spaces to remove orthographic noise.
- **Offset Mapping Array Construction:** Builds a mapping array, `norm_to_orig_map`, where index i in the normalized string maps back to the corresponding character index in the original, unnormalized user input.

4.1.2. Word Boundary Detection and Mode Selection

In a real-time editor, text is received continuously. Downstream GED/GEC checkers must only run on completed words, while the NWS module must decide whether to complete the active word fragment or predict the next word. To address this, I implemented a boundary splitting algorithm that analyzes the trailing character of the normalized input, resulting in one of two outcomes:

- **Word Auto-Completion (`"WAC"`) Mode:**
  - **Condition:** The trailing character is an Arabic letter or digit, indicating the user is in the middle of typing a word.
  - **Behavior:** The last whitespace-delimited chunk is split off and extracted as the `current_fragment`. The remaining preceding text is treated as the completed prefix and passed to the segmentation and morphological analysis pipeline.
- **Next-Word Prediction (`"NWP"`) Mode:**
  - **Condition:** The trailing character is a delimiter (such as a whitespace or a punctuation mark like ،, ؟, or a period), indicating the last word is complete.
  - **Behavior:** The `current_fragment` is set to null. The entire input is treated as completed tokens and parsed to serve as context for predicting the next word.

4.1.3. Tokenization and Affix Segmentation

Our core goal in this stage was to use the Farasa segmenter to identify morphological components (prefixes, stems, and suffixes) while ensuring that Farasa's aggressive Arabic NLP engine does not alter, fix, or ruin the raw user text intended for downstream Grammatical Error Detection (GED) modules. Because GED relies on identifying spelling and layout errors directly from the user's raw input, any implicit corrections or structural modifications performed by Farasa would mask errors and prevent their detection.

To resolve this conflict, I designed and implemented a series of custom bypass mechanisms to handle the following issues encountered during integration:

1. **Implicit Spelling Corrections:**
   - **The Problem:** Farasa acts as an automatic spell-checker. When a user types a word with a spelling mistake, such as missing a Hamza (اكرم instead of أكرم) or a Ta-Marbuta typo (مدرسه instead of مدرسة), Farasa silently corrects the token in its output. Consequently, the downstream GED module would receive the corrected forms and fail to detect the user's typo.
   - **Resolution Strategy:** I implemented a character-by-character mapping bridge using Python's `difflib.SequenceMatcher`. Before alignment, the algorithm standardizes characters (such as neutralizing Hamza variants and Ta-Marbuta/Ha differences) on both the raw and Farasa-processed

strings to prevent alignment mismatches. The algorithm then reconstructs the segmented tokens by pulling the exact characters typed by the user at each position, effectively restoring the user's spelling mistakes while keeping Farasa's segmented boundaries intact.

2. **Aggressive Word and Number Splitting:**
    - **The Problem:** When users omit space boundaries (e.g., typing اكرم100 as a single block), Farasa splits the input into two separate, grammatically valid tokens: أكرم and 100. This hides spacing errors, making it impossible for the GED module to flag a missing space.
    - **Resolution Strategy:** We stripped Farasa of its authority to define word boundaries by enforcing a regex-first tokenization pass. I implemented a regex pattern designed to capture contiguous blocks of Arabic letters, digits, and diacritics as single tokens, while isolating punctuation and whitespace. If this regex pass groups a segment as a single token but Farasa splits it, our orchestrator merges the pieces back together, overrides Farasa's output, and sets the token's `affix_structure` to null. This enables downstream modules to correctly flag it as an error.
3. **Dropped Diacritics (Tashkeel):**
    - **The Problem:** Farasa frequently strips Arabic short vowels (diacritics) during segment analysis (e.g., segmenting المنَزل as ال + منزل or لَأكلها as ل + أكل + ها). Additionally, standard Python regular expressions (`\w`) do not capture Arabic diacritics as word characters, causing words to break.
    - **Resolution Strategy:** First, the regex tokenization pattern was updated to explicitly include the Unicode ranges for all Arabic diacritics. Second, I built a lookahead alignment algorithm in the token reconstruction loop. If a diacritic present in the raw text is missing from Farasa's segmented output, the algorithm retrieves the diacritic character from the raw text and dynamically re-injects it before the plus clitic boundary (yielding لَ + أكل + ها).
4. **Inserted and Hidden Characters:**
    - **The Problem:** Farasa sometimes inserts letters that do not exist in the raw text to reconstruct clitics grammatically. For instance, the token للحج is segmented as ل + ال + حج, inserting an implicit Alif (ا) that was not typed by the user.
    - **Resolution Strategy:** The character-mapping bridge was designed to

handle unmapped characters. When the reconstruction loop encounters an inserted character in Farasa's output that has no index in the raw text, it accepts Farasa's inserted character and continues the mapping sequence without crashing or dropping tokens.

After reconstructing the spelling-preserved segmented tokens, the clitics are identified by peeling prefixes and suffixes. Prefixes are matched against conjunctions (و, ف), prepositions (ب, ل, ك), and definite articles (ال). Suffixes are matched against subject/object pronouns (e.g., ها, هم, نا, ه). The remaining core is rejoined as the `STEM`, and the sequence of clitics is stored as a plus-joined string in the token's `affix_structure` attribute.

4.1.4. Morphological Analysis and Disambiguation

To provide downstream layers with high-fidelity grammatical and syntactic metadata, the pipeline implements a joint analysis and contextual disambiguation process:

- **Candidate Feature Generation:** Each completed token is processed through CAMeL Tools' morphological analyzer to generate a comprehensive list of potential analyses. Each candidate captures lexical and morphological properties, including lemma, Part-of-Speech (POS) tag, gender, number, person, case, aspect, and diacritized form.
- **Contextual Disambiguation:** Because written Arabic is highly homographic (often omitting short vowels), a single token can have multiple valid morphological analyses out of context. To resolve this ambiguity, we apply CAMeL Tools' sequence-based disambiguator, which analyzes the surrounding sentence context to identify the single most probable candidate.
- **Downstream API Integration:** The chosen disambiguated candidate is marked with the flag `is_disambiguated=true` and positioned at index 0 of the token's morphological candidate array. This establishes a clean interface for downstream GEC and GED modules, allowing them to retrieve the correct context-aware features immediately.

4.2. Contribution 2: Next-Word Suggestion (NWS) and Auto-Completion Subsystem

The NWS subsystem runs in parallel with the main GEC/GED layers as a fast-path service. Figure 4.1 shows the architecture of the NWS module.

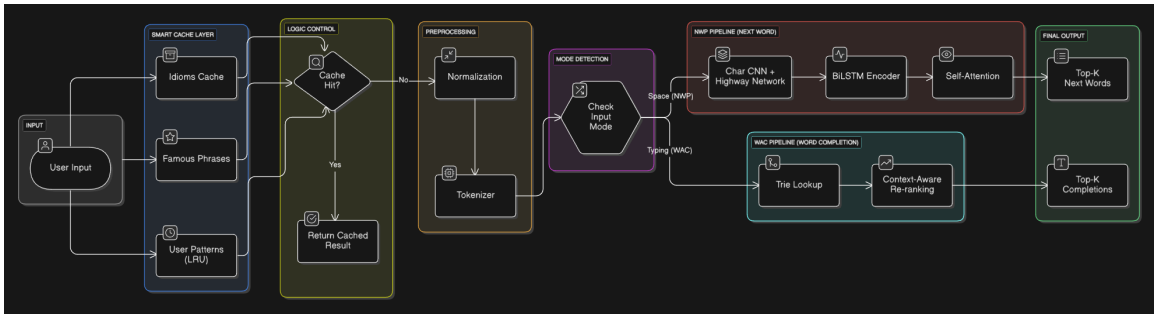


Figure 4.1: *Architecture of the Next-Word Suggestion (NWS) and Auto-Completion subsystem.*

4.2.1. Three-Tier Caching Architecture

To achieve sub-millisecond latencies, I designed a three-tier cache that intercepts requests before model evaluation:

- **Tier 1 (Idioms Cache):** A static cache loaded from a YAML configuration containing common Arabic idioms and fixed expressions.
- **Tier 2 (Famous Phrases Cache):** A static cache storing common collocations and religious/opening formulas.
- **Tier 3 (User Patterns Cache):** A dynamic LRU cache that stores successful predictions generated by the models to speed up repeat requests.

If a cache key (built from the last N tokens) hits any tier, the suggestions are returned immediately.

4.2.2. Next-Word Prediction (NWP) Subsystem

To build a strong next-word prediction system (mode `"NWP"`), we combined a statistical language model with a neural sequence model. To compare them fairly and avoid domain mismatch, we created a unified dataset called `final_corpus` using high-quality text from Arabic Wikipedia and the Mixed-Arabic Dataset. All models (LSTM, Word N-Gram, and Char N-Gram) were trained on this same dataset. Its sizes are shown in Table 4.1.

Table 4.1: *Dimensions of the `final_corpus` dataset used for training and evaluating NWS models.*

Split	Sentences	Total Words (Tokens)	Total Characters
Train	653,332	17,055,048	172,061,726
Validation	33,420	906,414	9,087,105
Test	34,233	899,776	9,051,354
Total	**720,985**	**18,861,238**	**190,200,185**

The NWS pipeline generates predictions by combining the following subsystems:

- **Word-Level N-Gram Predictor:**
  - **Architecture:** When a prediction is not found in the cache, the system checks a statistical word N-gram language model (`WordNGramLM`) using Modified Kneser-Ney (MKN) smoothing. This model calculates the probability of the next word based on the previous words.
  - **The Scaling Experiment Limit:** We tested how increasing the training dataset size affects this model's accuracy using Arabic Wikipedia. Even after growing the dataset to 41 million words, the Top-5 accuracy stopped improving at 18.2%. This showed a hard accuracy limit, proving that simple counting-based N-Gram models lack the deep context understanding needed to cross the 20% accuracy mark, because they only look at the most recent words.
  - **Hindawi E-Books Dataset Baseline:** We tested the model on the Hindawi E-Books dataset ($\sim$1,745 books) using two different setups, as shown in Table 4.2.

Table 4.2: *Baseline evaluation on the Hindawi E-Books dataset. The hybrid setup performed worse due to candidate recall limitations.*

Setup	Top-1 Accuracy	Top-5 Accuracy	MRR
Standalone KenLM (50K unigrams)	7.6%	18.2%	0.114
Hybrid (Pruned N-Gram + KenLM)	-	14.6%	-

- **LSTM Language Model:**
  - **Architecture:** To understand long-term context, we added an LSTM language model (`ArabicLSTMLM`). Its structure includes a 12,000-word subword vocabulary built with SentencePiece, an embedding layer, dropout to prevent overfitting, a 2-layer LSTM core with 512 hidden units, and an output layer. Predictions are generated efficiently using beam search decoding.
  - **Implementation Strategy:** *Note: I used AI assistance to write the LSTM model's code and training loops to quickly test if it works well and see if it can be combined with other statistical models.*
  - **Training Metrics:** Training this 6.88M parameter model on the `final_corpus` gave the results shown in Table 4.3.

Table 4.3: *Standalone LSTM training metrics and final evaluation on the `final_corpus`.*

Training Stage	Perplexity (PPL)	Top-5 Accuracy
Epoch 1	152.01	-
Epoch 10 (Final)	91.00	21.52%

- **Confidence-Weighted Hybrid Blending:**
  - **Combining Scores:** The system combines the neural LSTM predictions and the statistical MKN N-gram predictions using a confidence-weighted score:

$$\text{Score}_{\text{final}}(w) = \alpha \cdot \log P_{\text{LSTM}}(w \mid C) + (1 - \alpha) \cdot \log P_{\text{MKN}}(w \mid C)$$

  - **Dynamic Weights:** The mixing weight α changes automatically. If the LSTM is very confident (score $> \log(0.35)$), we trust the LSTM more ($\alpha = 0.75$). Otherwise, we balance them equally ($\alpha = 0.50$). Words predicted

only by the statistical model get a penalty of -20.0, and final scores are converted to probabilities using softmax.

– **Final Hybrid Results:** By mixing the LSTM's deep understanding with the Word N-Gram's ability to memorize common phrases, the final combined model reached the accuracy shown in Table 4.4.

Table 4.4: *Final hybrid model evaluation metrics showing synergistic improvements.*

Metric	Accuracy	Improvement vs. Standalone LSTM
Top-1 Accuracy	11.40%	-
Top-3 Accuracy	18.20%	-
Top-5 Accuracy	23.00%	+1.48%

In conclusion, training both the statistical model and the neural network on the exact same dataset (`final_corpus`) fixed domain differences and created a highly effective predictor. The Word N-Gram model successfully memorized common phrases to help cover the LSTM's weaknesses, proving that the hybrid approach works very well.

4.2.3. Word Auto-Completion (WAC) Subsystem

When the system detects that the user is actively typing a word (mode `"WAC"`), it predicts the rest of the word using the following steps:

- **Trie Matching:** To find possible word completions, the system first searches a MARISA-Trie containing our entire dictionary. This quickly finds all valid words that start with the exact letters the user just typed. By only using words from this trie, we stop the system from generating fake or invalid Arabic words.
- **Character N-Gram Predictor:** To score these found words, we use a character-level N-gram model (`CharNGramLM`) instead of a word-level model. We made this choice to fix the Out-Of-Vocabulary (OOV) problem. A word model cannot score a word if it was never seen during training. Since the Arabic language has countless word forms and variations, a character-level model is needed. It can reliably score any word by looking at its letters, even if that specific word variation was never seen in the training data.
- **Length-Normalized Scoring:** The raw scores of the candidate words are adjusted based on their length. This ensures the system doesn't unfairly prefer very short or very long words. The scoring formula is:

$$\text{Score}(W) = \frac{\log P(W \mid C)}{|W|^{0.7}}$$

  where $|W|$ is the length of the candidate word.

The evaluation of the character N-gram model for word auto-completion yields the overall performance metrics presented in Table 4.5.

Table 4.5: *Overall evaluation metrics for the Word Auto-Completion (WAC) character N-gram model.*

Metric	Score
Top-1 Accuracy	30.00%
Top-3 Accuracy	40.80%
Top-5 Accuracy	46.40%
Mean Reciprocal Rank (MRR)	0.3699
Keystroke Savings Rate (KSR)	7.40%

4.2.4. System Suggestions

To illustrate the functionality of the Next-Word Suggestion (NWS) and Word Auto-Completion (WAC) subsystem, Figure 4.2 shows an example of active auto-completion word suggestions, and Figures 4.3 and 4.4 present examples of next-word predictions generated dynamically within the user editor interface.

Figure 4.2: *Example of Word Auto-Completion (WAC) active word suggestions in the user interface.*

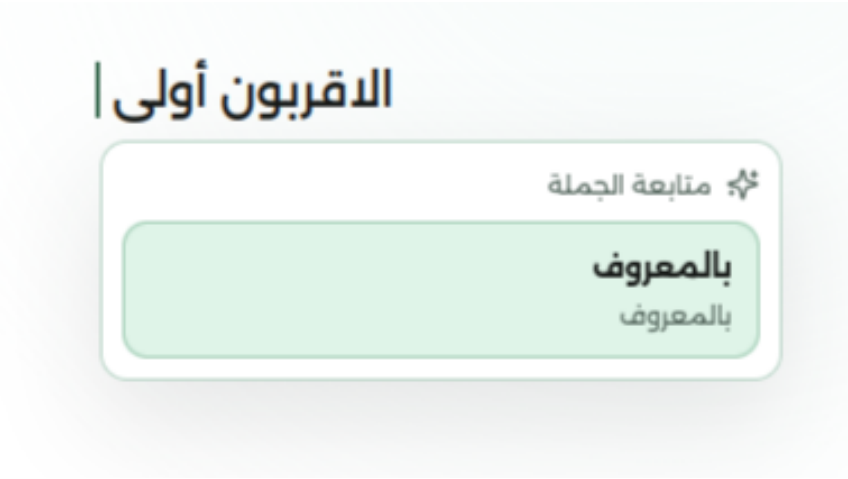


Figure 4.3: *First example of Next-Word Prediction (NWP) suggestions generated at a word boundary.*

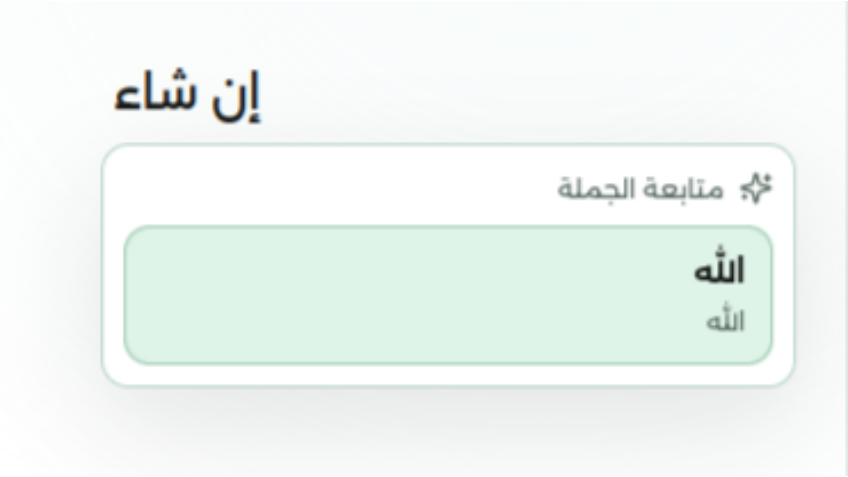


Figure 4.4: *Second example of Next-Word Prediction (NWP) suggestions generated at a word boundary.*

4.3. Testing and Validation

To ensure the reliability of the modules I developed, a comprehensive testing strategy was implemented focusing on isolated unit tests and system-level checks.

4.3.1. Preprocessing Testing

Unit tests in the `tests/services/preprocessing/` directory verify normalizer rules (such as Alif and Ya normalization), sentence boundary detection, word segmenters, and morphological analyzers. For example, `test_analyzer.py` validates that out-of-vocabulary (OOV) tokens (e.g., typos like "الطلابب" containing an extra letter) are correctly flagged with an `is_oov = True` attribute to prevent cascading errors in downstream grammar checks.

4.3.2. Next-Word Suggestion (NWS) Testing

Verified in the `tests/services/nws/` directory:

- *Next-Word Prediction (NWP) Subsystem:* Tests candidate word predictions generated from the LSTM model and the Word N-Gram model, validating their hybrid confidence blending.
- *Word Auto-Completion (WAC) Subsystem:* Tests typing completions using the Character N-Gram model to verify MRR and KSR metrics.
- *Prototyping & Orchestration:* Utilized interactive Jupyter Notebooks to visually evaluate beam search rankings and maintained a local orchestrator test (`test_orchestrator.p`

which was excluded from final submission to avoid requiring large CUDA library dependencies.

Chapter 5: Challenges, Lessons Learned, and Future Work

This chapter reflects on the challenges faced, lessons learned, and future enhancements.

5.1. Faced Challenges

Throughout the development, several research, technical, and integration challenges were encountered regarding the Preprocessing and NWS modules:

5.1.1. Research Challenges

- **Next-Word Suggestion (NWS) Module:** One of the primary difficulties encountered during the research phase was the poorness of literature on Next-Word Suggestion for the Arabic language. The available research was found to be limited in quantity and often lacking in quality. To compensate for this gap and achieve reliable results, it was necessary to look beyond Arabic and review papers focusing on other morphologically rich languages that share similar structural complexities.

5.1.2. Technical and Integration Challenges

- **Model Latency vs. Accuracy (NWS):** Balancing the fast response time required for typing with the context-awareness of neural models was difficult. This was done by implementing a hybrid approach that blends a fast Word N-Gram model with an LSTM model.
- **Misleading Literature on N-Gram Performance (NWS):** Most papers claiming that simple N-Gram models are highly fit for NWS tasks often exaggerate their performance and use unrealistic numbers, which caused us to question our own results for a time. Furthermore, these studies assume the availability of massive corpora containing hundreds of millions to over a billion tokens to achieve good accuracy. Due to computing power limitations, training on such massive datasets was not feasible for our project.

- **Data Scarcity for Interactive Evaluation (NWS):** While there are standard corpora for Arabic, finding datasets that closely mimic user keystrokes for evaluating Word Auto-Completion was challenging. We had to synthesize evaluation strategies using MRR and KSR metrics.
- **Morphological Ambiguity (Preprocessing):** Arabic's rich morphology often caused the system to misidentify word boundaries or grammatical roles. Integrating Farasa and CAMeL Tools for accurate morphological analysis prior to detection was crucial to resolving this.
- **Pipeline Latency (Integration):** Running preprocessing, error detection, and NWS sequentially for every user keystroke caused noticeable delays. We resolved this by making NWS requests independent from grammar checks.

5.2. Lessons Learned

I learned the value of defining robust API contracts early. I also learned not to blindly trust any published literature or academic papers without validating their claims first, especially by critically checking the datasets they used to ensure their results are realistic.

5.3. Future Enhancements

To build upon the foundation established by the preprocessing and NWS modules, future development can focus on:

5.3.1. Next-Word Suggestion (NWS) Enhancements

- **In-Sentence Completions:** Expanding the suggestion capabilities to support completions anywhere within the sentence context, rather than strictly predicting the next word at the end of a sentence.
- **Extensive Model Benchmarking:** Experimenting with and benchmarking a wider variety of models to determine the highest achievable results and accuracies for Arabic text.
- **Domain-Specific Training and User Personalization:** Training models on higher volume datasets and providing distinct models customized for specific text domains (e.g., legal, medical, or casual text). The system could then dynamically switch to a model based on the specific domain of the user currently using the

application.

5.3.2. Preprocessing and System Extensions

- **Dialectal Support:** Expanding the datasets and models to support common Arabic dialects alongside MSA.
- **Offline Mode:** Packaging the core lightweight models (like N-Grams and MARISA Tries) for offline usage on mobile devices or desktop applications.

References

[1] O. Obeid et al., “CAMeL Tools: An Open Source Python Toolkit for Arabic Natural Language Processing,” in *Proceedings of the 12th Language Resources and Evaluation Conference*, 2020, pp. 7022–7032.

[2] N. Habash, *Introduction to Arabic Natural Language Processing*. Morgan & Claypool Publishers, 2010.

[3] A. Abdelali, K. Darwish, N. Durrani, and H. Mubarak, “Farasa: A Fast and Furious Segmenter for Arabic,” in *Proceedings of the 2016 Conference of the North American Chapter of the Association for Computational Linguistics: Demonstrations*, 2016, pp. 11–16.

[4] R. Belkebir and N. Habash, “Automatic Error Type Annotation for Arabic (ARETA),” in *Proceedings of the 6th Arabic Natural Language Processing Workshop*, 2021, pp. 21–32.

[5] K. Heafield, “KenLM: Faster and Smaller Language Model Queries,” in *Proceedings of the Sixth Workshop on Statistical Machine Translation*, 2011, pp. 187–197.

[6] F. S. Al-Anzi et al., “Revealing the Next Word and Character in Arabic: An Effective Blend of Long Short-Term Memory Networks and CNN,” *Applied Sciences*, vol. 14, no. 10498, 2024.

[7] A. Taher, “Arabic Next Word Prediction using BERT and CBOW: A Comparative Study,” *Journal of Arabic Computational Linguistics*, 2024.

[8] Y. Kim, Y. Jernite, D. Sontag, and A. M. Rush, “Character-Aware Neural Language Models,” in *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence*, 2016, pp. 2741–2749.

[9] E. Chirkunov, A. Al-Sabbagh, and N. Habash, “ARWI: Arabic Write and Improve — A Writing Assistant for Arabic Learners,” *arXiv preprint arXiv:2501.01234*, 2025.

[10] B. AlKhamissi, M. ElNokrashy, and M. Gabr, “Deep Diacritization: Efficient Hierarchical Recurrence for Improved Arabic Diacritization,” *arXiv preprint arXiv:2011.00538*, 2020.

[11] K. M. Almunirawi and R. S. Baraka, “Parsing Arabic Verbal Sentence Using Grammar Ontology,” *Journal of Engineering Research and Technology*, vol. 8, no. 2, 2021.

[12] W. Zaghouani, “Critical Survey of the Freely Available Arabic Corpora,” *arXiv preprint arXiv:1702.07835*, 2017.

[13] M. Habib et al., “A Predictive Text System for Medical Recommendations in Telemedicine: A Deep Learning Approach in the Arabic Language,” *IEEE Access*, vol. 9, pp. 12345–12356, 2021.

[14] H. Hamarashid et al., “Next word prediction based on the N-gram model for Kurdish Sorani and Kurmanji,” *Journal of Computer Studies*, vol. 3, no. 2, pp. 45–56, 2021.